

EduTrack: Complete Full-Stack Validations Map

This document exhaustively breaks down every data validation layer implemented across the entire EduTrack ecosystem. Validations are handled in a 3-tier architecture: **Frontend (UI checks)**, **Backend (API checks)**, and **Database (Strict schema constraints)**.

1. Top Tier: Frontend Validations (Client-Side)

These rules are processed inside your users' browsers. They provide immediate feedback to users, blocking bad data from even attempting to reach the server.

Real-Time Input Filtering

- **File:** `frontend/src/components/StudentRegistrationForm.tsx` & `frontend/src/components/LoginPage.tsx`
- **Rule:** When typing into the phone number box, any characters that aren't numbers (like letters, `+`, or `-`) are instantly stripped out, and the string is hard-capped to 10 digits as the user types.

Pre-Submission Blockers (Submit Button Validations)

- **File:** `frontend/src/components/StudentRegistrationForm.tsx` & `frontend/src/components/StudentList.tsx`
- **Rule:** Before allowing a "Create" or "Update" action, the code verifies the `parent_email` strictly matches standard email format (`/^[^\s@]+@[^\s@]+\.[^\s@]+$/`) and `parent_phone` is exactly 10 digits (`/^\d{10}$/`). Triggers a red "Validation Error" if failed.

Complex Business Logic constraints (The "Bucket Rule")

- **File:** `frontend/src/components/GradeManagement.tsx`
 - **Rule:** Before enrolling a student into a subject, the application actively scans their current enrollment array. It blocks teachers from assigning 2 subjects in the same academic bucket (e.g., selecting both "Art" and "Drama" if they are in the same elective category tree) via `showValidationError` .
-

2. Middle Tier: Backend Validations (Server-Side)

This is the application's secondary defense. Even if someone bypasses the frontend (e.g. using Postman), the backend routes reject maliciously crafted or incomplete requests.

Centralized Middleware Interceptors

- **File:** `backend/src/middleware/validation.ts`
- **Rule:** `validateStudentRegistration()` loops through 13 required fields across student and parent columns to verify they aren't blank. It re-verifies the 10-digit phone constraint, forces `first_name` to be `length > 1`, and verifies email regex.
- **Rule:** `validateGradeCreation()` ensures custom grade definitions have valid numerical components (restricted logically between grade 6 and 13) and class sections (e.g., "Science A") match an alphanumeric structure under 30 characters.

Per-Route Controller Validations

- **File:** `backend/src/routes/marks.ts` (and other route files like `students.ts` , `subjects.ts` , etc.)
- **Rule:** Validates dynamic URL parameters. Checks if expected numbers are passed (`isNaN(id)`) and refuses the request with `status(400) Invalid ID` .

- **Rule: Mark Ceiling / Floor Validation:** Specifically checks if `marks_obtained` is between 0 and `max_marks` . It handles exceptions only for the designated "AB" (Absent) status string, otherwise rejecting the entry as impossible.
 - **Rule:** Validates Authorization Context (`req.user.role === 'teacher'`). Refuses 403 Access Denied if a teacher attempts to input/edit marks for a class ID they are not directly assigned to.
-

3. Base Tier: Database Constraints (SQL-Level Schema)

The ultimate absolute source of truth. Even if an engineer mistakenly disables the frontend and backend checks, the MySQL database schema natively blocks invalid formats trying to save.

Type Boundaries and ENUM Sets

- **File:** Database generation logic and patches like `backend/src/migrations/update-exam-types.sql` .
- **Rule:** Enforces strict specific lists. For example, `exam_type` is physically constrained in the database columns as `ENUM('first', 'second', 'third') NOT NULL` .
- **Rule:** `NOT NULL` : Physically restricts required columns from accepting blank entries.
- **Rule:** Structural dependencies: `user_id` and `teacher_id` linkages prevent the deletion or editing of records if the originating teacher is removed without resolving the orphans first.